

JAVA SDE-2 INTERVIEW PREPARATION SERIES

DATA STRUCTURES AND ALGORITHMS - BOOK 12 OF 17 - STUDY STEP DSA-12

Binary Search: Bounds, Answers, and Invariants

Exact Matches, Lower Bounds, Monotone Predicates, and Safe Midpoints

BY

Vinay Reddy Kalluri

Stop memorizing one template: define the candidate interval, monotone predicate, progress rule, and boundary contract that make binary search correct.

BOUNDS | INVARIANTS | LOWER BOUND | ROTATION | ANSWER SEARCH

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Updated 2026-08-02

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

60-second entry rule: If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to [DSA 11 - Stacks, Queues, and Deques](#). If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

Binary search is a proof pattern over monotone state. This volume centers interval semantics, duplicates, insertion points, overflow-safe midpoint calculation, and search on answers.

Readiness map

Decision	Evidence
START HERE	The search space is sorted or governed by a monotone predicate; A first, last, lower, or upper boundary is required; Trying an answer can be checked efficiently; Each comparison can eliminate a contiguous region.
PREPARE FIRST	JAVA-01 and DSA-01 through DSA-06; Complexity and loop invariants.
MOVE ON WHEN	Choose closed or half-open interval templates; Implement lower bounds and insertion points; Use Java <code>binarySearch</code> APIs without misreading negative results; Prove progress and handle duplicates safely.

Choose the route that fits today

- 1 **Learning:** read in order, type the representative Java examples, and complete each dry run.
- 2 **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

DSA Segment Position

DSA 11 - Stacks, Queues, and Deques	YOU ARE HERE DSA 12 - Binary Search	DSA 13 - Trees, BSTs, and Tries
-------------------------------------	---	---------------------------------

A note from Vinay

Binary search is not about finding a middle element; it is about preserving a boundary contract. Write what is known on each side before you choose the update rule.

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION	
Start Here - Your Route Through This Volume	2
Part I - Core Learning	4
Chapter 1 - Binary Search Foundations: Search Space, Invariant, and Progress	4
Chapter 2 - Binary Search: Bounds, Monotone Answers, and Invariants for SDE-2	8
Chapter 3 - Essential Advanced Binary-Search Pattern Clinics	22
Chapter 4 - Realistic Binary Search Interview Rounds	26
Chapter 5 - Boundary Engineering and Search on Answers	30
Chapter 6 - Searching Continuous and Unimodal Domains	35
Chapter 7 - Binary Search Practice Lab	42
Chapter 8 - Binary Search Practice Lab Solutions	44
Chapter 9 - Executable Binary-Search Boundary Checks	46
Part II - Practice and Handoff	54

Part I - Core Learning

SDE-2 CORE CHAPTER

Chapter 1 - Binary Search Foundations: Search Space, Invariant, and Progress

Binary search is safe only when a comparison or predicate lets you discard one contiguous part of the remaining search space. Sorted data is the familiar case; a monotone yes/no predicate is the general case.

Begin with exact search

For sorted `[2, 5, 8, 12, 16]`, searching for 12 compares the middle value 8. Because every value left of 8 is also too small, the entire left half can be discarded.

Use a closed interval when both endpoints are candidates:

JAVA

```
static int indexOf(int[] sorted, int target) {
    int left = 0;
    int right = sorted.length - 1;
    while (left <= right) {
        int middle = left + (right - left) / 2;
        if (sorted[middle] == target) {
            return middle;
        }
        if (sorted[middle] < target) {
            left = middle + 1;
        } else {
            right = middle - 1;
        }
    }
    return -1;
}
```

The invariant is: if the target exists, an occurrence remains inside `[left, right]`. The updates exclude `middle`, which has already been proven unequal, so the candidate interval strictly shrinks.

Why the midpoint formula matters

JAVA

```
int middle = left + (right - left) / 2;
```

For nonnegative valid indexes, `left + right` can overflow while `right - left` cannot. Parentheses also make the intended arithmetic explicit.

Closed versus half-open

Convention	Candidates	Loop	Empty when
Closed	[left, right]	left <= right	left > right
Half-open	[left, right)	left < right	left == right

Do not combine a closed-loop condition with half-open updates from memory. Write what each boundary means.

WHY IT WORKS | Lower bound from first principles

The lower bound is the first index whose value is at least the target. It is also the insertion position before existing equal values.

JAVA

```
static int lowerBound(int[] sorted, int target) {
    int left = 0;
    int right = sorted.length;
    while (left < right) {
        int middle = left + (right - left) / 2;
        if (sorted[middle] < target) {
            left = middle + 1;
        } else {
            right = middle;
        }
    }
    return left;
}
```

Invariant:

- all indexes before `left` are known to contain values less than target;
- all indexes at or after `right` are known to contain values at least target;
- `[left, right)` remains unknown.

For `[1, 2, 2, 2, 5]`, target 2, the result is 1. For target 4, the result is 4. For target 9, the result is `length`.

Upper bound changes the comparison to discard values less than or equal to target. The count of target in sorted data is `upperBound - lowerBound`.

First true: the general template

Suppose predicate results across ordered candidates are:

TEXT

```
false false false true true true
```

Lower bound is exactly "first true" for predicate `value >= target`. Many SDE-2 answer-search problems have the same shape: capacity, speed, days, distance, or threshold.

Before searching an answer, prove:

- 1 a bounded candidate domain;
- 2 a monotone predicate;
- 3 which boundary is requested;
- 4 a predicate cost that makes the total worthwhile.

Termination clinic

The common infinite loop is retaining `middle` without changing interval size:

JAVA

```
// Broken for some two-element intervals  
left = middle;
```

If `middle == left`, no progress occurs. Use `left = middle + 1` when `middle` is excluded, or choose an upper-biased midpoint for a last-true template that intentionally retains `middle`.

Duplicates change the question

Exact search may return any matching index. First occurrence, last occurrence, first greater-or-equal, and first strictly greater are distinct contracts. Say the boundary aloud before code.

Library semantics

`Arrays.binarySearch` returns an index when found. Otherwise it returns `-(insertionPoint) - 1`. Recover the insertion point with `-result - 1`. With duplicates, the returned matching index is not promised to be the first.

Complexity

After k comparisons, at most roughly $n / 2^k$ candidates remain, so exact and boundary searches use $O(\log n)$ comparisons and $O(1)$ auxiliary space. Searching an answer with a predicate costing $O(f(n))$ costs $O(f(n) \log R)$, where R is the integer candidate-range size.

TRY IT | Foundation checkpoint

- 1 What does each boundary mean in your exact-search template?
- 2 Why must an unsuccessful lower bound be allowed to return `length`?
- 3 How do duplicates change exact search versus first occurrence?
- 4 What proves an answer predicate is searchable?
- 5 Show that every update strictly reduces your interval.

SDE-2 CORE CHAPTER

Chapter 2 - Binary Search: Bounds, Monotone Answers, and Invariants for SDE-2

Binary search is a proof pattern, not a remembered loop. The searchable object is an ordered decision boundary: all states on one side have one classification and all states on the other side have another. Sometimes the ordered domain is a sorted array. Sometimes it is a numeric capacity, time, rate, or precision interval whose feasibility predicate is monotone. At SDE-2 level, define the interval and invariant aloud, make every branch preserve them, and explain duplicates, overflow, and library return semantics.

CHOOSE IT | Recognition map

Prompt signal	Search target	Preferred template
find an exact value in sorted data	one matching index	classic exact search
first position not less than x	insertion boundary	lower bound
first position greater than x	insertion boundary	upper bound
first/last occurrence	duplicate range	lower/upper composition
smallest feasible capacity/time/rate	monotone false-to-true boundary	first true
largest feasible budget/threshold	monotone true-to-false boundary	last true or last false after inversion
array rotated from sorted order	one ordered half per iteration	rotated search
row/column order or flattened matrix order	ordered coordinates	matrix-specific binary search
local peak exists	slope points toward a peak	neighbor comparison
real-valued approximation	continuous interval	iteration/tolerance search

Do not binary-search merely because the input is an array or because the answer is numeric. You need a total order or a monotone predicate. If feasibility changes **false**, **true**, **false**, discarding half the domain is unsound.

Pick one interval convention

This chapter favors half-open intervals `[low, high)` for bounds. They represent an empty interval naturally when `low == high`, and `high` may be the array length as a valid insertion sentinel. For exact search it uses a closed interval `[low, high]`, because success terminates immediately and the empty condition `low > high` is familiar. Mixing update rules across these templates is a common source of nontermination.

Overflow-safe midpoint

For nonnegative indexes where `low <= high`, compute:

TEXT

```
mid = low + (high - low) / 2
```

not `(low + high) / 2`, whose sum may overflow. For a `long` answer domain, even `high - low` can overflow if the interval spans nearly the entire signed range. Most interview domains are nonnegative and bounded by input sums; state that assumption or use a representation/unsigned technique suited to the full range.

Family 1: exact search

Invariant and proof

For a sorted ascending array, use a closed candidate interval. The invariant is:

If the target exists in an unexamined position, at least one occurrence lies in `[low, high]`.

Compare `a[mid]` with target. Equality returns. If `a[mid] < target`, sortedness proves indexes `<= mid` cannot match, so assign `low = mid + 1`. Otherwise assign `high = mid - 1`. Each unsuccessful step removes `mid`, shrinking the interval. When `low > high`, no candidate remains.

Dry-run target 7 in `[1, 3, 5, 7, 9]`: `[0, 4]` examines 5, leaving `[3, 4]`; it examines 7 and returns index 3. Searching 6 follows `[0, 4] -> [3, 4] -> [3, 2]`, then returns absent.

Time is $O(\log n)$, auxiliary space $O(1)$. With duplicates, exact search may return any occurrence. If the contract requires first or last, use a boundary search rather than storing a match and improvising.

Family 2: lower bound, upper bound, first, and last

Lower bound

`lowerBound(a, x)` returns the first index whose value is at least `x`, or `a.length`. Maintain `[low, high)` with:

- all indexes before `low` have value `< x`;
- all indexes at or after `high` have value `>= x` (within the array);
- the boundary lies in `[low, high]`.

If $a[mid] < x$, move `low` to `mid+1`; otherwise move `high` to `mid`. At equality of boundaries, `low` is the first legal insertion point.

Upper bound

`upperBound(a, x)` returns the first index whose value is greater than x . The only comparison change is that values $\leq x$ move `low` right. Therefore occurrences occupy half-open range $[lowerBound, upperBound)$, count is `upper - lower`, first is lower when in range, and last is `upper - 1` when a match exists.

Dry-run $[1, 2, 2, 2, 4]$, target 2 : lower bound converges to 1 ; upper bound converges to 4 . The first/last range is indexes $1..3$, and insertion before equals uses 1 while insertion after equals uses 4 .

Bounds handle empty arrays and edge insertion naturally. They also compose with range-counting and sorted deduplication. Do not access `a[bound]` before checking `bound < a.length`.

Family 3: first true and last false

Many "binary search on answer" problems reduce to a monotone predicate over integer candidates:

TEXT

```
false false false true true true
                ^ first true
```

Use $[low, high)$ and a virtual true sentinel at `high` if no real candidate succeeds. The invariant is that candidates below `low` are known false and candidates at/above `high` are known true or sentinel. If `predicate(mid)` is true, retain it by setting `high=mid`; otherwise discard it with `low=mid+1`. The return may equal the original high-exclusive value, meaning no true candidate.

`lastFalse` is `firstTrue - 1` when the false region may be empty. That sentinel is not representable when `firstTrue` is `Integer.MIN_VALUE`; the sample uses exact subtraction and rejects that case instead of wrapping. A production API can return an optional boundary when an empty false region is valid. If the requested pattern is true-to-false, invert the predicate or derive an explicit last-true template and prove its progress. Avoid memorizing mirrored updates without a boundary model.

The predicate's cost matters: total time is $O(P * \log R)$, where P is one predicate evaluation and R is numeric range width. Predicate calls must not mutate shared state in a way that changes later truth values.

Family 4: rotated arrays, including duplicates

For a strictly increasing array rotated once, at least one half around `mid` is sorted. If $a[low] \leq a[mid]$, the left half is sorted. Check whether target lies within its ordered bounds; keep that half or discard it. Otherwise the right half is sorted and the symmetric test applies.

Duplicates can hide which half is informative: with $a[low] == a[mid] == a[high]$, either side may contain the pivot or target. Shrink both endpoints. This preserves correctness but can degrade worst-case time to $O(n)$, as in an array of mostly equal values. Be explicit: binary search does not guarantee logarithmic time when duplicates erase ordering information.

Dry-run `[2, 5, 6, 0, 0, 1, 2]`, target `0`: middle index `3` matches immediately. For target `3`, branches eliminate ordered ranges and eventually return false. On `[1, 0, 1, 1, 1]`, endpoint shrinking may be required before the sorted half becomes visible.

Family 5: peak finding

A peak is an element not smaller than its neighbors under the problem's exact definition. For the common strict adjacent-inequality version, compare `a[mid]` with `a[mid+1]` while searching `[low, high]`. If the slope descends (`a[mid] > a[mid+1]`), a peak exists at `mid` or left, so set `high=mid`. If it ascends, a peak exists to the right, so set `low=mid+1`.

Why can a half be discarded? Following an ascending slope right cannot fall off the array without encountering a peak; following a descending slope left has the symmetric guarantee. The candidate interval always contains a peak and shrinks until one index remains. Time $O(\log n)$, space $O(1)$. Duplicate plateaus require a changed peak definition or may destroy the strict directional argument.

Family 6: matrix search

If every row is sorted and the first value of each row is greater than the previous row's last value, the matrix is globally sorted in row-major order. Treat virtual index `i` as `(i / columns, i % columns)` and run exact binary search. Validate rectangular shape. Compute total cells with checked multiplication; coordinate arithmetic must not overflow.

For `[[1, 3, 5], [7, 9, 11]]`, virtual indexes `0..5` map to the sorted sequence. Searching `9` examines virtual `2` (`5`), then `4` (`9`). Complexity is $O(\log(\text{rows} * \text{cols}))$, space $O(1)$.

If rows and columns are sorted independently but row ranges overlap, flattening is invalid. Use the top-right staircase: too large moves left, too small moves down, giving $O(\text{rows} + \text{cols})$. State which matrix contract you have.

Family 7: monotone answer search

Suppose packages in fixed order must be shipped within `days`. For capacity `C`, simulate days required. If `C` is feasible, every larger capacity is feasible; if it is infeasible, every smaller one is infeasible. This is the monotonicity proof.

The smallest capacity lies between the maximum single weight and the sum of weights. Use `long`: a sum can overflow `int`. Predicate invariant: current load never exceeds `C`; when the next package would exceed it, start a new day, preserving order. Stop early when days used exceeds the limit.

Dry-run weights `[3, 2, 2, 4, 1, 4]`, days `3`. Lower bound is `4`, upper `16`. Capacity `10` needs two days, so search left. Capacity `7` needs three and is feasible. Capacity `5` needs four and fails. Capacity `6` needs three, so the minimum is `6`.

Total cost is $O(n \log(\text{sum} - \text{max} + 1))$, auxiliary space $O(1)$. Answer search is often the right SDE-2 move when constructing the optimum directly is hard but checking a proposed answer is simple.

Precision search over real values

Real-valued binary search cannot rely on `low+1`. Stop after a fixed iteration count, on an absolute/relative interval tolerance, or both. Fixed iterations provide a deterministic error contraction; after `k` steps, interval width is initial width divided by 2^k . For IEEE-754 `double`, around 100 iterations is generally beyond useful precision for ordinary ranges.

For square root of nonnegative `x`, predicate `mid*mid >= x` risks overflow. Compare `mid >= x/mid` for positive `mid`, or control the interval and special cases. Define behavior for NaN, infinities, negative input, signed zero, and acceptable error. If exact integer root is requested, use integer arithmetic and division-based comparisons instead.

Java library semantics

`Arrays.binarySearch(array, key)` returns some matching index if found. If absent, it returns `-(insertionPoint) - 1`, where insertion point is the first position at which the key could be inserted while preserving order. Recover it with `int insertion = -result - 1` only when `result < 0`.

With duplicates, the returned matching index is not promised to be first or last. The array/range must already be sorted under the same ordering. Object overloads and `Collections.binarySearch` depend on comparator consistency; sorting with one comparator and searching with another invalidates the precondition.

Complete Java 21 reference implementation

Compile and run with `java -ea BinarySearchSde2`.

JAVA (continued 1/6)

```
import java.util.Arrays;
import java.util.OptionalInt;
import java.util.function.IntPredicate;

public final class BinarySearchSde2 {
    private BinarySearchSde2() {}

    public static int exactSearch(int[] sorted, int target) {
        requireArray(sorted);
        int low = 0, high = sorted.length - 1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (sorted[mid] == target) return mid;
            if (sorted[mid] < target) low = mid + 1;
            else high = mid - 1;
        }
        return -1;
    }

    public static int lowerBound(int[] sorted, int target) {
        requireArray(sorted);
        int low = 0, high = sorted.length;
        while (low < high) {
            int mid = low + (high - low) / 2;
            if (sorted[mid] < target) low = mid + 1;
            else high = mid;
        }
        return low;
    }

    public static int upperBound(int[] sorted, int target) {
        requireArray(sorted);
        int low = 0, high = sorted.length;
        while (low < high) {
            int mid = low + (high - low) / 2;
            if (sorted[mid] <= target) low = mid + 1;
        }
    }
}
```


JAVA (continued 2/6)

```
        else high = mid;
    }
    return low;
}

public static OptionalInt firstIndex(int[] sorted, int target) {
    int index = lowerBound(sorted, target);
    return index < sorted.length && sorted[index] == target
        ? OptionalInt.of(index) : OptionalInt.empty();
}

public static OptionalInt lastIndex(int[] sorted, int target) {
    requireArray(sorted);
    int index = upperBound(sorted, target) - 1;
    return index >= 0 && sorted[index] == target
        ? OptionalInt.of(index) : OptionalInt.empty();
}

public static int firstTrue(int lowInclusive, int highExclusive,
    IntPredicate monotonePredicate) {
    if (lowInclusive > highExclusive || monotonePredicate == null) {
        throw new IllegalArgumentException("invalid range or predicate");
    }
    int low = lowInclusive, high = highExclusive;
    while (low < high) {
        int mid = (int) (low + ((long) high - low) / 2);
        if (monotonePredicate.test(mid)) high = mid;
        else low = mid + 1;
    }
    return low;
}

public static int lastFalse(int lowInclusive, int highExclusive,
    IntPredicate monotonePredicate) {
    return Math.subtractExact(
```

JAVA (continued 3/6)

```
        firstTrue(lowInclusive, highExclusive, monotonePredicate), 1);
    }

    public static boolean searchRotatedWithDuplicates(int[] values, int target) {
        requireArray(values);
        int low = 0, high = values.length - 1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (values[mid] == target) return true;
            if (values[low] == values[mid] && values[mid] == values[high]) {
                low++;
                high--;
            } else if (values[low] <= values[mid]) {
                if (values[low] <= target && target < values[mid]) high = mid - 1;
                else low = mid + 1;
            } else {
                if (values[mid] < target && target <= values[high]) low = mid + 1;
                else high = mid - 1;
            }
        }
        return false;
    }

    public static int peakIndex(int[] values) {
        requireArray(values);
        if (values.length == 0) throw new IllegalArgumentException("empty array");
        int low = 0, high = values.length - 1;
        while (low < high) {
            int mid = low + (high - low) / 2;
            if (values[mid] > values[mid + 1]) high = mid;
            else low = mid + 1;
        }
        return low;
    }
}
```

JAVA (continued 4/6)

```
public static boolean searchRowMajorMatrix(int[][] matrix, int target) {
    int columns = validateRectangular(matrix);
    if (matrix.length == 0 || columns == 0) return false;
    int cells = Math.multiplyExact(matrix.length, columns);
    int low = 0, high = cells - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        int value = matrix[mid / columns][mid % columns];
        if (value == target) return true;
        if (value < target) low = mid + 1;
        else high = mid - 1;
    }
    return false;
}

public static long minimumShipCapacity(int[] weights, int days) {
    requireArray(weights);
    if (weights.length == 0 || days <= 0) {
        throw new IllegalArgumentException("nonempty weights and positive days required");
    }
    long low = 0, high = 0;
    for (int weight : weights) {
        if (weight <= 0) throw new IllegalArgumentException("weights must be positive");
        low = Math.max(low, weight);
        high = Math.addExact(high, weight);
    }
    while (low < high) {
        long mid = low + (high - low) / 2;
        if (canShip(weights, days, mid)) high = mid;
        else low = mid + 1;
    }
    return low;
}

private static boolean canShip(int[] weights, int allowedDays, long capacity) {
```

JAVA (continued 5/6)

```
    int usedDays = 1;
    long load = 0;
    for (int weight : weights) {
        if (load + weight > capacity) {
            usedDays++;
            load = 0;
            if (usedDays > allowedDays) return false;
        }
        load += weight;
    }
    return true;
}

public static double squareRoot(double value, double relativeTolerance) {
    if (!Double.isFinite(value) || value < 0
        || !(relativeTolerance > 0) || !Double.isFinite(relativeTolerance)) {
        throw new IllegalArgumentException("finite nonnegative value/tolerance required");
    }
    if (value == 0 || value == 1) return value;
    double low = 0, high = Math.max(1.0, value);
    for (int iteration = 0; iteration < 200; iteration++) {
        double mid = low + (high - low) / 2.0;
        if (mid >= value / mid) high = mid;
        else low = mid;
        if (high - low <= relativeTolerance * Math.max(1.0, high)) break;
    }
    return (low + high) / 2.0;
}

private static void requireArray(int[] values) {
    if (values == null) throw new IllegalArgumentException("array is null");
}

private static int validateRectangular(int[][] matrix) {
    if (matrix == null) throw new IllegalArgumentException("matrix is null");
}
```

JAVA (continued 6/6)

```

    if (matrix.length == 0) return 0;
    if (matrix[0] == null) throw new IllegalArgumentException("null row");
    int columns = matrix[0].length;
    for (int[] row : matrix) {
        if (row == null || row.length != columns) {
            throw new IllegalArgumentException("matrix must be rectangular");
        }
    }
    return columns;
}

public static void main(String[] args) {
    int[] data = {1, 2, 2, 2, 4, 7, 9};
    assert exactSearch(data, 7) == 5 && exactSearch(data, 6) == -1;
    assert lowerBound(data, 2) == 1 && upperBound(data, 2) == 4;
    assert firstIndex(data, 2).orElseThrow() == 1;
    assert lastIndex(data, 2).orElseThrow() == 3;
    assert firstTrue(0, 10, x -> x * x >= 30) == 6;
    assert lastFalse(0, 10, x -> x >= 6) == 5;
    assert firstTrue(Integer.MIN_VALUE, Integer.MAX_VALUE, x -> x >= 0) == 0;

    assert searchRotatedWithDuplicates(new int[] {2, 5, 6, 0, 0, 1, 2}, 0);
    assert !searchRotatedWithDuplicates(new int[] {2, 5, 6, 0, 0, 1, 2}, 3);
    int[] mountain = {1, 3, 5, 4, 2};
    assert peakIndex(mountain) == 2;

    int[][] matrix = {{1, 3, 5}, {7, 9, 11}};
    assert searchRowMajorMatrix(matrix, 9) && !searchRowMajorMatrix(matrix, 8);
    assert minimumShipCapacity(new int[] {3, 2, 2, 4, 1, 4}, 3) == 6;
    assert Math.abs(squareRoot(2, 1e-12) - Math.sqrt(2)) < 1e-10;

    int library = Arrays.binarySearch(data, 6);
    assert library < 0 && -library - 1 == 5;
}
}

```

Complexity and contract matrix

Operation	Time	Space	Contract caveat
exact/lower/upper bound	$O(\log n)$	$O(1)$	input sorted under same order
first true	$O(\log R)$ predicate calls	$O(1)$	predicate monotone on domain
rotated with duplicates	average $O(\log n)$, worst $O(n)$	$O(1)$	one rotation of nondecreasing data
peak	$O(\log n)$	$O(1)$	adjacent/peak definition supports slope proof
row-major matrix	$O(\log(rc))$	$O(1)$	rectangular and globally row-major sorted
minimum capacity	$O(n \log S)$	$O(1)$	positive weights, fixed ordering
real square root	$O(\log(\text{range/error}))$ or capped	$O(1)$	floating-point error contract

R is discrete range size and S is the capacity search interval. State predicate cost rather than calling every answer search simply $O(\log n)$.

WATCH OUT | Edge cases and common mistakes

- Empty arrays: exact search is absent; lower and upper bound are zero.
- Duplicates: arbitrary match, first, and last are different contracts.
- Half-open templates use `high=mid`; closed templates often use `high=mid-1`. Do not mix them.
- A loop that assigns `low=mid` can stall when two candidates remain. Bias the midpoint or use first-true form.
- Check a returned insertion index before array access.
- Sorting order and comparator used for search must agree.
- Rotated duplicates may force linear work; do not claim unconditional logarithmic time.
- A matrix sorted only by rows cannot be flattened into one sorted sequence.
- Sum-based upper bounds and products need `long` or exact arithmetic.
- Feasibility must be monotone and preferably side-effect-free.
- Floating-point search needs tolerance/iteration and special-value policy, not equality with a computed real.

TRY IT | Exercises with model checkpoints

Exercise 1: count values in a range

Count sorted-array values in inclusive numeric range $[a, b]$.

Checkpoint: if $a > b$, return zero. Answer is `lowerBound(first > b)` minus `lowerBound(first >= a)`, i.e. `upperBound(b) - lowerBound(a)`. The half-open index interval makes duplicates automatic.

Exercise 2: search a rotated array for first physical occurrence

The array contains duplicates and the contract asks for the smallest index containing target.

Checkpoint: ordinary rotated search can prove existence but not first physical index without more work. Discuss worst-case $O(n)$ and whether finding the pivot plus bound searches is valid under duplicates. Optimize only after making the index contract explicit.

Exercise 3: minimum eating rate

Find the smallest integer rate that finishes piles within h hours.

Checkpoint: rate domain is $[1, \text{maxPile}]$; hours at rate r is sum of ceilings $(\text{pile} + r - 1) / r$, computed overflow-safely as $(\text{pile} - 1) / r + 1$ for positive piles. Hours is nonincreasing, so feasibility `hours <= h` is false-to-true as rate grows. Stop the sum early above h .

Exercise 4: integer square root

Return floor square root for a nonnegative `long`.

Checkpoint: search for last `m` with `m * m <= x/m`, handling zero separately. Do not test `m*m<=x`, which can overflow. Use a closed or boundary template with a documented upper limit.

Exercise 5: matrix contract comparison

Implement both row-major search and top-right staircase search.

Checkpoint: provide a matrix sorted by every row and column that violates global row-major order. Show that flattening can discard the target incorrectly, while staircase search uses column/row monotonicity correctly.

Exercise 6: test a monotone predicate

Build a property test for an answer-search predicate.

Checkpoint: for sampled `x < y`, assert that `predicate(x)` implies `predicate(y)` for false-to-true monotonicity. This cannot prove all inputs, but catches stateful predicates and arithmetic errors. Separately compare binary results to a linear oracle on small randomized domains.

SDE-2 production follow-ups

How do you search remote or expensive data? The number of comparisons is logarithmic, but each comparison may be a network round trip or disk read. Batch/prefetch nearby metadata, cache immutable results, enforce deadlines, and reconsider whether an index service should expose a direct bound operation.

Can the collection change during search? Mutation can invalidate sortedness and indexes. Use immutable snapshots, version checks, locks, or a data structure with defined concurrent navigation. Visibility alone does not preserve a multi-step search invariant.

How do you prove an answer predicate? Separate necessity and sufficiency. Show smaller candidates cannot work when current fails, and larger candidates cannot fail when current works. Then audit overflow and state mutation, because either can make observed truth nonmonotone even when the mathematics is monotone.

How do you test boundaries? Compare against a linear oracle on small arrays; generate empty, singleton, all-equal, minimum/maximum integer, target-below, target-above, duplicate-run, rotated, and no-solution cases. Assert both returned result and postconditions such as insertion partition.

WHY IT WORKS | Interview proof clinic and model answers

Why does lower bound set `high=mid` rather than `mid-1`? When `a[mid] >= target`, `mid` itself may be the first qualifying index and must remain a candidate. The half-open interval permits retaining it by moving the exclusive upper boundary to `mid`. When `a[mid] < target`, `mid` cannot qualify, so `low=mid+1` discards it. These updates preserve the partition invariant and always shrink the interval.

How do you know first-true returns the minimum feasible answer? At termination `low==high`. The invariant says everything below `low` is proven false, and `high` is proven true or is the no-answer sentinel.

Therefore no smaller feasible candidate exists, and if the result is not the sentinel it is feasible. This proves minimality, not only that some feasible value was found.

Can a stateful predicate break search? Yes. If caching, rate limits, random choices, overflow, or mutation changes its answer between calls, the observed sequence may not be monotone. Isolate state, use immutable inputs, make arithmetic wide/exact enough, and test predicate monotonicity separately from the search loop.

When is interpolation or exponential search relevant? Exponential search finds an upper boundary when the sorted domain is conceptually unbounded: probe `1, 2, 4, . . .` until crossing, guarding overflow, then binary-search the bracket. Interpolation search estimates a position from values but depends on distribution and can degrade badly; ordinary binary search is the dependable baseline.

How would you search a versioned remote API? Pin all predicate/element reads to one immutable version. Otherwise concurrent updates can change the ordering between comparisons. Count network round trips, apply deadlines and retries only when safe, and prefer a server-side lower-bound endpoint because asymptotically few calls can still be operationally expensive.

What is the right answer when no candidate is feasible? Encode it explicitly: optional result, sentinel outside the legal domain, or exception for violated preconditions. The first-true helper returns `highExclusive` as a sentinel, but a domain API should translate that into a type the caller cannot mistake for a real capacity. Test all-false and all-true domains.

A complete boundary-review checkpoint

Before submitting, write three examples beside the loop: empty domain, one candidate, and two candidates. Trace whether the midpoint and update remove or retain the examined candidate. Then assert the postcondition, not merely one expected index: every value before lower bound is smaller and every value at/after it is not smaller; every capacity below the returned minimum is infeasible and the returned capacity is feasible. These partition assertions expose bugs that favorable examples miss. For a costly predicate, cache only if candidates and input version fully determine the result; otherwise stale state can violate the proof. Finally, translate the generic sentinel into a domain result before returning from a public API.

Final readiness checklist

- I state the domain, ordering, interval convention, and invariant first.
- Every branch discards `mid` or safely retains a boundary while guaranteeing progress.
- I distinguish exact match, lower bound, upper bound, first, and last.
- I prove predicate monotonicity before answer search.
- I include predicate cost and range width in complexity.
- I audit duplicates, sentinels, arithmetic overflow, and library encoding.
- For matrices and peaks, I use the actual structural guarantee-not an imagined global sort.

Binary search becomes reliable when every assignment is justified by a boundary proof. Once that habit is learned, capacities, deadlines, rates, and precision problems become the same disciplined search in different clothing.

SDE-2 CORE CHAPTER

Chapter 3 - Essential Advanced Binary-Search Pattern Clinics

Most binary-search questions search an index or a monotone answer. Two advanced variants search a partition or an ordered numeric domain that is not materialized as an array.

Clinic 1: median of two sorted arrays

Partition instead of merge

Partition both arrays so the combined left side contains half the elements. Binary-search only the shorter array. If its cut is `cutA`, the other cut is determined:

TEXT

```
cutB = (lengthA + lengthB + 1) / 2 - cutA
```

The partition is valid when:

TEXT

```
leftA <= rightB and leftB <= rightA
```

If `leftA > rightB`, cut A too far right. Otherwise `leftB > rightA`, so cut A too far left. Sentinels represent missing values at array boundaries.

For odd total length, the median is the maximum left value. For even length, average the maximum left and minimum right values using `long` before addition.

Why the shorter array matters

Searching the shorter array keeps `cutB` within valid bounds and gives $O(\log(\min(m,n)))$ time. The input arrays are not modified, and auxiliary space is $O(1)$.

Clinic 2: kth smallest in a row-and-column sorted matrix

The matrix values define an ordered answer domain even though flattening it would destroy the desired memory bound. Binary-search value `x` using this monotone predicate:

TEXT

```
count(values <= x) >= k
```

Count in $O(\text{rows} + \text{columns})$ by starting at the bottom-left. If the current value is at most x , every value above it in that column also qualifies; add $\text{row} + 1$ and move right. Otherwise move up.

Duplicates are handled naturally because the predicate counts occurrences. The total complexity is $O((\text{rows} + \text{columns}) \log(\text{valueRange}))$, which should be compared with a heap solution when the numeric range is very wide or k is small.

Runnable Java 21 clinic

JAVA (continued 1/3)

```
import java.util.Objects;

public final class BinarySearchCoverageClinic {
    private BinarySearchCoverageClinic() {}

    public static double medianOfTwoSortedArrays(int[] first, int[] second) {
        Objects.requireNonNull(first, "first");
        Objects.requireNonNull(second, "second");
        if (first.length + second.length == 0) {
            throw new IllegalArgumentException("at least one value is required");
        }
        if (first.length > second.length) {
            return medianOfTwoSortedArrays(second, first);
        }

        int low = 0;
        int high = first.length;
        while (low <= high) {
            int cutFirst = low + (high - low) / 2;
            int cutSecond = (first.length + second.length + 1) / 2 - cutFirst;

            int leftFirst = cutFirst == 0 ? Integer.MIN_VALUE : first[cutFirst - 1];
            int rightFirst = cutFirst == first.length
                ? Integer.MAX_VALUE : first[cutFirst];
            int leftSecond = cutSecond == 0 ? Integer.MIN_VALUE : second[cutSecond - 1];
            int rightSecond = cutSecond == second.length
                ? Integer.MAX_VALUE : second[cutSecond];

            if (leftFirst <= rightSecond && leftSecond <= rightFirst) {
                int bestLeft = Math.max(leftFirst, leftSecond);
                if ((first.length + second.length) % 2 == 1) {
                    return bestLeft;
                }
                int bestRight = Math.min(rightFirst, rightSecond);
            }
        }
    }
}
```

JAVA (continued 2/3)

```
        return ((long) bestLeft + bestRight) / 2.0;
    }
    if (leftFirst > rightSecond) {
        high = cutFirst - 1;
    } else {
        low = cutFirst + 1;
    }
}
throw new IllegalArgumentException("inputs must be sorted");
}

public static int kthSmallestInSortedMatrix(int[][] matrix, int k) {
    int columns = validateRectangular(matrix);
    long total = (long) matrix.length * columns;
    if (k < 1 || k > total) {
        throw new IllegalArgumentException("k is outside the matrix");
    }

    long low = matrix[0][0];
    long high = matrix[matrix.length - 1][columns - 1];
    while (low < high) {
        long middle = low + (high - low) / 2;
        if (countAtMost(matrix, columns, middle) >= k) {
            high = middle;
        } else {
            low = middle + 1;
        }
    }
    return (int) low;
}

private static long countAtMost(int[][] matrix, int columns, long target) {
    int row = matrix.length - 1;
    int column = 0;
```

JAVA (continued 3/3)

```
        long count = 0;
        while (row >= 0 && column < columns) {
            if (matrix[row][column] <= target) {
                count += row + 1L;
                column++;
            } else {
                row--;
            }
        }
        return count;
    }

    private static int validateRectangular(int[][] matrix) {
        if (matrix == null || matrix.length == 0 || matrix[0] == null
            || matrix[0].length == 0) {
            throw new IllegalArgumentException("matrix must be nonempty");
        }
        int columns = matrix[0].length;
        for (int[] row : matrix) {
            if (row == null || row.length != columns) {
                throw new IllegalArgumentException("matrix must be rectangular");
            }
        }
        return columns;
    }

    public static void main(String[] args) {
        assert medianOfTwoSortedArrays(new int[] {1, 3}, new int[] {2}) == 2.0;
        assert medianOfTwoSortedArrays(new int[] {1, 2}, new int[] {3, 4}) == 2.5;
        int[][] matrix = {{1, 5, 9}, {10, 11, 13}, {12, 13, 15}};
        assert kthSmallestInSortedMatrix(matrix, 8) == 13;
        System.out.println("PASS essential binary-search clinics");
    }
}
```

Expected output with assertions enabled:

TEXT

PASS essential binary-search clinics

Interviewer follow-up chain with model answers

Interviewer: Why is merging not the target solution for the median problem?

Candidate: Merge is correct and a good baseline, but it costs $O(m+n)$ time. The sorted boundary condition lets us search a partition in $O(\log(\min(m,n)))$ time and $O(1)$ space.

Interviewer: Does the matrix algorithm require all values to be distinct?

Candidate: No. It counts occurrences less than or equal to the candidate, so duplicate values occupy multiple ranks correctly.

SDE-2 CORE CHAPTER

Chapter 4 - Realistic Binary Search Interview Rounds

Round 1: first and last position

Prompt

In a sorted array, return the first and last index of `target`, or `[-1, -1]` when absent.

Candidate answer

Derive both from boundaries instead of modifying an exact-search loop. `first = lowerBound(target)`. `after = upperBound(target)`. The target is absent when `first == length` or `sorted[first] != target`.

JAVA

```
static int[] equalRange(int[] sorted, int target) {  
    int first = lowerBound(sorted, target);  
    if (first == sorted.length || sorted[first] != target) {  
        return new int[] {-1, -1};  
    }  
    return new int[] {first, upperBound(sorted, target) - 1};  
}
```

Why not search left and then scan? Scanning can become $O(n)$ when many duplicates exist. Two boundary searches remain $O(\log n)$.

Overflow in `target + 1`? Do not implement upper bound by lower-bounding `target + 1`; `Integer.MAX_VALUE + 1` wraps. Implement the strict comparison directly.

Round 2: search a rotated sorted array

Prompt

Search a rotated array of distinct integers in $O(\log n)$.

Model reasoning

At least one half around the middle is normally sorted. Determine which half, then decide whether target lies inside its value range.

JAVA

```
static int searchRotated(int[] values, int target) {
    int left = 0;
    int right = values.length - 1;
    while (left <= right) {
        int middle = left + (right - left) / 2;
        if (values[middle] == target) {
            return middle;
        }
        if (values[left] <= values[middle]) {
            if (values[left] <= target && target < values[middle]) {
                right = middle - 1;
            } else {
                left = middle + 1;
            }
        } else {
            if (values[middle] < target && target <= values[right]) {
                left = middle + 1;
            } else {
                right = middle - 1;
            }
        }
    }
    return -1;
}
```

Follow-up answers

What if duplicates are allowed? When `values[left] == values[middle] == values[right]`, sorted-half identification can be ambiguous. Shrinking both ends preserves correctness but can degrade worst-case time to $O(n)$.

What invariant remains? If target exists, it remains in the closed candidate interval. The sorted-half test justifies which contiguous side can be removed.

Round 3: minimum ship capacity

Prompt

Packages with positive integer weights must be shipped in order within `days`. Return the minimum daily capacity.

Clarification and bounds

Capacity cannot be smaller than the heaviest package and never needs to exceed the total weight. Feasibility is monotone: if capacity **C** works, every larger capacity works.

JAVA (continued 1/2)

```
static long minimumCapacity(int[] weights, int days) {
    if (weights.length == 0 || days <= 0) {
        throw new IllegalArgumentException("invalid input");
    }
    long left = 0L;
    long right = 0L;
    for (int weight : weights) {
        if (weight <= 0) {
            throw new IllegalArgumentException("weights must be positive");
        }
        left = Math.max(left, weight);
        right += weight;
    }
    while (left < right) {
        long middle = left + (right - left) / 2L;
        if (canShip(weights, days, middle)) {
            right = middle;
        } else {
            left = middle + 1L;
        }
    }
}
```

JAVA (continued 2/2)

```
    }
    return left;
}

static boolean canShip(int[] weights, int days, long capacity) {
    int usedDays = 1;
    long load = 0L;
    for (int weight : weights) {
        if (load + weight > capacity) {
            usedDays++;
            load = 0L;
        }
        load += weight;
        if (usedDays > days) {
            return false;
        }
    }
    return true;
}
```

Follow-up answers

Why long? The total of valid positive `int` weights can overflow `int`.

Why greedily fill each day? For fixed capacity and order, delaying a package cannot reduce the number of days. Filling until the next package no longer fits minimizes days for that capacity.

Complexity? $O(n \log S)$, where **S** is the candidate capacity range from maximum weight to total weight; $O(1)$ auxiliary space.

Closing answer pattern

State interval semantics, boundary meaning, monotonicity proof, midpoint bias, progress proof, empty/duplicate behavior, numeric width, and predicate cost.

SDE-2 CORE CHAPTER

Chapter 5 - Boundary Engineering and Search on Answers

Binary search is not "look at the middle until found." It is maintaining a boundary between two regions whose truth is known. Once the interval convention and predicate are explicit, exact lookup, lower/upper bounds, rotated arrays, matrix search, and capacity optimization become variations of one proof.

The complete Java 21 implementations are in `BinarySearchInterviewChecks.java`.

Prefer a half-open insertion boundary

Lower bound returns the first index whose value is at least target. Search `[left, right)`, initially `[0, n)`:

TEXT

```
[0, left)   values < target
[right, n)  values >= target
[left, right) unknown
```

At `middle`:

- if `values[middle] < target`, discard through middle: `left = middle + 1`;
- otherwise, middle may be the first valid index: `right = middle`.

When `left == right`, no unknown index remains and that boundary is the answer. It may equal `n`, which is a valid insertion position but not a valid array index.

Upper bound changes only equality: it finds the first value strictly greater than target. The half-open equal range is `[lowerBound, upperBound)`, and count is their difference.

Why loop updates must exclude something

Every branch must shrink the unknown interval. In a half-open first-true search, `right = middle` is safe because middle remains a candidate while width decreases; `left = middle + 1` excludes a proven-false middle.

Mixing a `while (left < right)` loop with `right = middle - 1` often skips candidates. Mixing inclusive and exclusive meanings is the source of most "almost works" binary searches.

Write the invariant above the loop before code.

Midpoint overflow

For nonnegative array indexes:

JAVA

```
int middle = left + (right - left) / 2;
```

avoids `left + right` overflow. A fully signed `long` domain can make even `high - low` overflow. The companion's reusable `firstTrue` uses an overflow-safe signed average:

JAVA

```
(low & high) + ((low ^ high) >> 1)
```

Most interview answer domains are nonnegative, where the ordinary difference formula is clearer. Use the stronger formula only when the broader contract needs it.

Search on answer

Some prompts do not provide a sorted array. Instead, candidate answers have a monotonic feasibility predicate:

TEXT

```
capacity too small: false false false
capacity sufficient:      true true true
                        first true is optimum
```

Shipping capacity uses lower bound `max(weight)` and upper bound `sum(weights)`. "Can ship within D days at capacity C" becomes no harder as C grows. The binary search finds the first feasible capacity.

Before searching, prove:

- 1 the answer domain contains the optimum;
- 2 feasibility is monotonic;
- 3 the chosen endpoint is guaranteed feasible; and
- 4 the predicate does not overflow.

For eating speed, hours per pile use ceiling division:

TEXT

```
(pile + speed - 1) / speed
```

with promotion to `long` before addition.

Integer square root without multiplication overflow

To test whether `middle * middle <= value`, use:

TEXT

```
middle == 0 || middle <= value / middle
```

The companion searches for the last valid middle and correctly handles `Long.MAX_VALUE`, whose floor square root is `3_037_000_499`.

This is a boundary problem: true values form a prefix and false values a suffix. Returning the saved last-true answer avoids confusing it with first-true code.

Rotated distinct array

In a sorted array rotated once, at least one half around middle is sorted when values are distinct.

TEXT

```
[4,5,6,7,0,1,2]
left half sorted when values[left] <= values[middle]
```

If target lies inside the sorted half's inclusive/exclusive value boundaries, keep that half; otherwise keep the other. With duplicates, `left`, `middle`, and `right` can be equal and reveal no sorted side. A duplicate-tolerant variant may shrink endpoints, degrading to $O(n)$ worst case. State which contract you implement.

Flattened matrix search

A rectangular matrix is searchable as one sorted sequence only if:

- each row is sorted; and
- the first value of each row is greater than the last of the previous row.

Map flat index `p` to `row = p / columns`, `column = p % columns`. Compute total cells in `long` before multiplying row and column counts. A matrix with rows sorted independently but overlapping ranges needs a different strategy.

Edge-case matrix

Case	Correct handling	Common failure
empty sorted array	lower/upper bound return zero	inspect index zero
target below all	boundary zero	return missing sentinel only
target above all	boundary <code>n</code>	dereference insertion position
many duplicates	lower and upper equality differ	arbitrary matching index
full signed long domain	overflow-safe midpoint	<code>high - low</code> overflow
answer high not feasible	reject/expand bound	return a false endpoint
predicate not monotonic	do not binary search	relying on samples
sum/capacity overflow	aggregate in <code>long</code>	overflow before assignment
square-root multiplication	compare by division	negative overflow looks feasible
rotated duplicates	use duplicate-aware contract	assuming one strict sorted half
empty matrix row	return false under rectangular contract	divide by zero
jagged matrix	reject or search rows separately	flat-index wrong row

Six live interview Q&A chains

1. Lower-bound result `n`

Interviewer: Your search returned array length. Is that a bug?

Candidate: Not for lower bound. It means every value is smaller and insertion belongs after the array. I check `index < n` before dereferencing for exact-match logic.

2. Equality branch

Interviewer: Why does lower bound move right on equality?

Candidate: Equality satisfies "at least target," but an earlier equal value may exist. Middle remains a candidate, so `right = middle`.

3. Answer predicate

Interviewer: How do you know shipping feasibility is monotonic?

Candidate: Increasing capacity never forces an additional day; the same packing remains possible or improves. Therefore false capacities precede true capacities.

4. High endpoint

Interviewer: Why use total weight as capacity high?

Candidate: With positive weights, that capacity ships everything in one day, so it is feasible for any positive day limit. A guaranteed true high is required by my first-true helper.

5. Rotated duplicates

Interviewer: Does your rotated search support duplicates?

Candidate: The companion intentionally requires distinct values. With equal left/middle/right, sorted-half identification is ambiguous; a robust duplicate version shrinks boundaries and can become linear.

6. Testing boundaries

Interviewer: How would you test lower and upper bound?

Candidate: Target before all, after all, equal at each edge, all duplicates, empty, and singleton. Then generate sorted duplicate-heavy arrays and compare both boundaries with linear scans. The companion runs that deterministic differential test.

Run the companion

BASH

```
javac --release 21 -Xlint:all -Werror BinarySearchInterviewChecks.java
java BinarySearchInterviewChecks
```

Expected final line: **PASS 18 binary-search checks.**

SDE-2 CORE CHAPTER

Chapter 6 - Searching Continuous and Unimodal Domains

Why this chapter exists

Every search in this volume so far has been over an integer index range, terminating when `low` meets `high`. Two escalations break that assumption, and both appear once an interviewer moves past array problems.

The domain is continuous. "Find the square root to six decimal places", "find the minimum radius covering every point". Integers do not converge to a point, and the `low < high` termination condition never fires. Floating-point search needs a different stopping rule, and the naive one loops forever.

The function is not monotonic. Binary search requires a predicate that is false then true, exactly once. A function that rises then falls has no such predicate - but it still has a findable extremum, via ternary search. Recognizing that a problem is *unimodal rather than monotonic* is the whole insight.

Part 1: search on real numbers

Why integer termination fails

Integer binary search terminates because the range shrinks by at least one each step and integers are discrete. On reals, $(low + high) / 2$ is strictly between them forever - or rather, until floating-point precision collapses the interval, which is an implementation accident rather than a designed stopping point.

Two correct stopping rules exist, and the second is the one to use.

Absolute tolerance. Stop when `high - low < epsilon`. Simple, and wrong for large magnitudes: an epsilon of `1e-9` is unreachable near `1e18`, because consecutive doubles there are further apart than that.

Fixed iteration count. Each step halves the interval, so `k` iterations reduce it by 2^{-k} . One hundred iterations shrink any starting range below any representable precision, and the loop is guaranteed to terminate.

JAVA

```

static double squareRoot(double target) {
    if (target < 0) {
        throw new IllegalArgumentException("negative input");
    }
    double low = 0;
    double high = Math.max(1, target);    // sqrt(x) <= x only when x >= 1
    for (int i = 0; i < 100; i++) {       // fixed count: always terminates
        double mid = low + (high - low) / 2;
        if (mid * mid < target) {
            low = mid;
        } else {
            high = mid;
        }
    }
    return low;
}

```

Three details:

Math.max(1, target) for the upper bound. For $\text{target} < 1$, the square root is *larger* than the target - $\text{sqrt}(0.25) = 0.5$ - so an upper bound of `target` excludes the answer. This is the most common bug in the problem, and it only shows on inputs below one.

low + (high - low) / 2 rather than (low + high) / 2. On integers this avoids overflow; on doubles it avoids losing precision when both endpoints are large and similar. Same habit, different reason.

A fixed 100 iterations, not a tolerance loop. It cannot hang, it needs no epsilon tuning, and 100 halvings exceed the precision of a `double` many times over. Interviewers recognize this as the robust choice.

Choosing the iteration count

Each iteration adds one bit of precision. A `double` has 52 mantissa bits, so around 60 to 70 iterations reach full precision for a starting range near 1. One hundred is a comfortable margin that costs nothing - the loop is 100 operations regardless of input size.

If the prompt asks for a specific tolerance, say six decimal places, you can compute it: from a range of width W , you need $\log_2(W / \text{epsilon})$ iterations. For $W = 1\text{e}18$ and $\text{epsilon} = 1\text{e}-6$, that is about 80. Stating the arithmetic rather than guessing is the signal.

Part 2: ternary search on unimodal functions

Monotonic versus unimodal

Binary search needs a predicate that switches from false to true exactly once. Formally, the sequence must be **monotonic** in that predicate.

A **unimodal** function has a single peak (or single valley) - it rises then falls, or falls then rises. There is no false-then-true predicate available, because "is this greater than the value to its right?" is false, then true, and the *boundary* is the peak. Actually that does give a valid predicate, which is why peak-finding by binary search works and is covered in this volume's family 5.

Ternary search is the alternative when comparing to a neighbour is not possible or not meaningful - typically on a continuous function you can only *evaluate*, not differentiate.

TEXT

```
monotonic:  F F F F T T T T    -> binary search the boundary
unimodal:   /\                  -> ternary search the peak
            /\
```

The method

Pick two interior points, evaluate both, and discard the third of the range that cannot contain the extremum.

JAVA

```
/** Maximum of a unimodal function on [low, high]. */
static double ternarySearchMax(DoubleUnaryOperator f, double low, double high) {
    for (int i = 0; i < 200; i++) {          // ternary shrinks slower; use more
        double leftThird = low + (high - low) / 3;
        double rightThird = high - (high - low) / 3;
        if (f.applyAsDouble(leftThird) < f.applyAsDouble(rightThird)) {
            low = leftThird;                  // peak is right of leftThird
        } else {
            high = rightThird;                // peak is at or left of rightThird
        }
    }
    return low + (high - low) / 2;
}
```

Why discarding is valid. If $f(\text{leftThird}) < f(\text{rightThird})$, the peak cannot lie left of `leftThird` - if it did, unimodality would force f to be decreasing across both points, making $f(\text{leftThird}) > f(\text{rightThird})$. So $[\text{low}, \text{leftThird}]$ is safe to discard. The symmetric argument covers the other case.

Unimodality is a precondition, not something the search checks. Run ternary search on a function with two peaks and it silently returns one of them, or neither. There is no error - just a wrong answer. Stating that precondition unprompted is the signal an interviewer wants.

The convergence rate. Each iteration keeps two thirds of the range, so k iterations shrink it by $(2/3)^k$. That is slower than binary search's $(1/2)^k$, and each iteration costs *two* function evaluations rather than one. Ternary search needs roughly 1.7 times as many iterations for the same precision, and about 3.4 times as many evaluations. Use it only when a monotonic predicate genuinely is not available.

Integer ternary search, and why to avoid it

On integers the two-point comparison breaks down when the range shrinks to two or three elements - `leftThird` and `rightThird` can coincide, and the loop stops making progress.

The reliable integer approach is to compare **adjacent** elements instead:

JAVA

```
// Peak finding: compare a[mid] with a[mid + 1] - this IS monotonic in the
// predicate "is the peak at or before mid?", so plain binary search works.
while (low < high) {
    int mid = low + (high - low) / 2;
    if (values[mid] < values[mid + 1]) {
        low = mid + 1;           // ascending: peak is to the right
    } else {
        high = mid;             // descending or peak: it is here or left
    }
}
return low;
```

This is the peak-finding template from family 5, and it is worth recognizing that **it is binary search, not ternary** - the adjacent comparison manufactures the monotonic predicate that unimodality alone did not provide. On integers, prefer this. Reserve ternary search for continuous domains where neighbours do not exist.

Part 3: binary search on the answer, on reals

The most valuable combination is applying real-valued binary search to a *feasibility* predicate - the "search the answer" pattern from family 3, but over a continuous parameter.

Given n houses on a line and k heaters, find the minimum radius so every house is covered.

The radius is continuous, and feasibility is monotonic in it: if radius r works, so does any larger radius. That monotonicity is what licenses binary search, and stating it is the load-bearing step.

JAVA

```
static double minimumRadius(double[] houses, double[] heaters) {
    double low = 0;
    double high = 1e9;           // any radius certainly sufficient
    for (int i = 0; i < 100; i++) {
        double mid = low + (high - low) / 2;
        if (covers(houses, heaters, mid)) {
            high = mid;           // feasible: try smaller
        } else {
            low = mid;            // infeasible: need larger
        }
    }
    return high;                 // return the feasible side
}
```

Return high, not low. `high` always holds a value known feasible; `low` always holds one known infeasible. On a minimisation the feasible side is the answer. Returning the wrong endpoint gives a result that fails the check by an epsilon - a bug that passes casual testing and fails an exact assertion.

This pattern generalizes widely: minimum time, minimum capacity, maximum minimum distance. The recipe is always the same - identify the continuous parameter, confirm feasibility is monotonic in it, write the feasibility check, and binary search a fixed number of iterations.

WATCH OUT | Edge cases and common mistakes

- Using `while (low < high)` on doubles, which never terminates.
- Using an absolute epsilon at large magnitudes where consecutive doubles are further apart than the epsilon.
- Bounding `sqrt(x)` above by `x`, which excludes the answer for `x < 1`.
- Computing the midpoint as `(low + high) / 2` and losing precision on large endpoints.
- Returning `low` from a minimisation search when `high` holds the feasible value.
- Applying ternary search to a function that is not unimodal; it returns a wrong answer silently.
- Using ternary search on integers, where the two probes can coincide and stall.
- Reaching for ternary search when an adjacent comparison gives a monotonic predicate and plain binary search suffices.
- Forgetting that ternary search costs two evaluations per iteration and converges more slowly.
- Failing to state the monotonicity of the feasibility predicate before searching the answer.

TRY IT | Interview questions and model answers

Find a square root to six decimal places.

Binary search on the real interval from 0 to `max(1, target)` - the upper bound must be at least 1 because for inputs below one the root is larger than the input. Run a fixed number of iterations, around 100, rather than looping on a tolerance: each iteration adds a bit of precision, the loop cannot hang, and no epsilon needs tuning.

Why not loop until `high - low < epsilon`?

It works for moderate magnitudes and fails for large ones, where consecutive representable doubles are further apart than the epsilon and the condition can never be satisfied. A fixed iteration count is bounded, precision-independent, and free.

When would you use ternary search?

On a continuous unimodal function where I can only evaluate, not compare neighbours - a single peak or valley with no monotonic predicate available. Each iteration keeps two thirds of the range and costs two evaluations, so it converges more slowly than binary search; I would only use it where a monotonic predicate genuinely does not exist.

Find the peak in an integer array. Ternary search?

No - binary search. Comparing `a[mid]` with `a[mid + 1]` manufactures a monotonic predicate: "is the peak at or before mid?" is false then true exactly once. Ternary search on integers also degenerates when the two probes coincide on a small range. Adjacent comparison is both correct and simpler.

What happens if the function is not actually unimodal?

Ternary search returns a wrong answer with no error. Unimodality is a precondition it does not verify, so it must be established from the problem rather than assumed.

In binary search on the answer, which endpoint do you return?

The one holding a known-feasible value - `high` for a minimisation, `low` for a maximisation. The other endpoint is known infeasible, so returning it produces a result that misses the requirement by an epsilon, which passes casual testing and fails an exact check.

TRY IT | Exercises

- 1 **Foundation:** Write real-valued binary search for a square root, then run it on 0.25 with an upper bound of `target` and observe the failure.
- 2 **Foundation:** Compute the iterations needed to reach `1e-9` precision from a starting range of `1e18`.
- 3 **Interview Core:** Replace the fixed loop with `while (high - low > 1e-9)` and find a magnitude where it does not terminate.
- 4 **Interview Core:** Implement ternary search for the maximum of a unimodal function and verify against a fine-grained scan.
- 5 **Interview Core:** Run ternary search on a two-peaked function and show it returns silently wrong output.
- 6 **Interview Core:** Solve peak finding with adjacent-comparison binary search, then attempt ternary search on integers and find where the probes coincide.
- 7 **SDE-2 Follow-up:** Measure function evaluations for binary and ternary search to identical precision and confirm the ratio.
- 8 **SDE-2 Follow-up:** Solve the heater-radius problem and assert the returned radius covers every house exactly; then return `low` instead and watch the assertion fail.
- 9 **SDE-2 Follow-up:** Take a maximisation "search the answer" problem and state which endpoint holds the feasible value.
- 10 **Challenge:** Given a function that is unimodal only within an unknown subrange, describe how you would locate that subrange before searching it.

RECAP | Chapter summary

Two escalations break the integer search this volume has assumed. On a continuous domain the `low < high` termination never fires, and the robust replacement is a fixed iteration count rather than a tolerance loop - each iteration adds a bit of precision, around 100 exceeds a double's mantissa many times over, and it cannot hang or need epsilon tuning at large magnitudes where an absolute tolerance becomes unreachable. The recurring traps are bounding `sqrt(x)` above by `x`, which excludes the answer below one, and returning the infeasible endpoint from an answer search when only the feasible side satisfies the requirement. The second escalation is a function that is unimodal rather than monotonic: ternary search finds its extremum by discarding a third per iteration, but converges more slowly than binary search and costs two evaluations per step, so it belongs only on continuous domains where no monotonic predicate exists. On integers, comparing adjacent elements manufactures exactly that predicate, which is why peak finding is binary search rather than ternary - and unimodality remains a precondition that ternary search never checks and silently violates when it fails.

TRY IT | Revision checklist

- ☐ I know why `low < high` cannot terminate on doubles.
- ☐ I use a fixed iteration count and can justify the number.
- ☐ I know why an absolute epsilon fails at large magnitudes.
- ☐ I bound `sqrt(x)` above by `max(1, x)` and know which inputs break otherwise.
- ☐ I compute midpoints as `low + (high - low) / 2` on reals as well as integers.
- ☐ I return the feasible endpoint from an answer search and know which one it is.
- ☐ I can distinguish monotonic from unimodal.
- ☐ I can justify why ternary search may discard a third of the range.
- ☐ I know ternary search costs two evaluations and converges more slowly.
- ☐ I use adjacent-comparison binary search for integer peak finding, not ternary.

SDE-2 CORE CHAPTER

Chapter 7 - Binary Search Practice Lab

- 1 Trace exact search for target 8 in [2, 5, 8, 12, 16].
- 2 Give lower and upper bounds of 2 in [1, 2, 2, 2, 5].
- 3 Explain why `while (left < right)` matches a half-open lower-bound interval.
- 4 Debug `middle = (left + right) / 2` for very large indexes.
- 5 Debug a loop that assigns `left = middle` with a lower-biased midpoint.
- 6 Recover insertion point from a negative `Arrays.binarySearch` result.
- 7 Implement integer square root without multiplication overflow.
- 8 Return the first index whose timestamp is at least a requested instant.
- 9 Find a peak element with a proof of the retained side.
- 10 Search a row-major sorted matrix without allocating a flattened copy.
- 11 Find the minimum eating speed or processing rate that meets a deadline.
- 12 Test whether a proposed feasibility predicate is actually monotone.
- 13 Explain how duplicates change rotated-array worst-case complexity.
- 14 Compare binary search on an array with search in a linked list.
- 15 Define a safe stopping rule for floating-point binary search.
- 16 **SDE-2 Follow-up:** Find the median of two sorted arrays by binary-searching a partition in the shorter array. Explain all four boundary values.
- 17 **SDE-2 Follow-up:** Find the kth smallest occurrence in a row-and-column sorted matrix by searching the value domain. Prove the `count(values <= middle)` predicate is monotone.

Boundary engineering lab

- 18 **Foundation:** Annotate lower bound with its two proven regions and dry-run target before, inside, and after the array.
- 19 **Interview Core:** Build a closed-domain first-true helper whose high endpoint must be true and whose midpoint is safe across signed `long` bounds.
- 20 **Interview Core:** Implement floor integer square root for every nonnegative `long` without multiplying the midpoint.
- 21 **Interview Core:** Search a rotated distinct array, then give a duplicate counterexample where sorted-half detection is ambiguous.
- 22 **Interview Core:** Search a row-major rectangular matrix through flat indexes computed in `long`; reject jagged input.
- 23 **SDE-2 Follow-up:** Differential-test lower/upper bounds against linear scans on random sorted duplicate-heavy arrays.

SDE-2 CORE CHAPTER

Chapter 8 - Binary Search Practice Lab Solutions

- 1 Start `[0, 4]`, middle 2, value 8, return index 2.
- 2 Lower bound is 1 and upper bound is 4; the count is 3.
- 3 `[left, right)` is nonempty exactly when `left < right`. Assigning `right = middle` retains middle as a candidate while excluding the old right side.
- 4 Use `left + (right - left) / 2`; for long answer domains use long arithmetic throughout.
- 5 When two candidates remain, lower-biased middle can equal left, so no progress occurs. Exclude middle with `left = middle + 1` or use an upper-biased last-true template.
- 6 `insertionPoint = -result - 1` when result is negative.
- 7 Search `x` over a bounded nonnegative domain and compare `middle <= value / middle` rather than `middle * middle <= value`; handle middle zero.
- 8 This is lower bound over ordered timestamps using predicate `timestamp >= requested`.
- 9 Compare `values[middle]` with `values[middle + 1]`. If rising, a peak exists to the right; otherwise one exists at or left of middle. Retain the proven side.
- 10 Map virtual index `i` to row `i / columns` and column `i % columns`; validate rectangular shape and use long only if total index arithmetic can overflow.
- 11 Bound the answer, define `canFinish(rate)`, prove that a faster rate remains feasible, then find first true.
- 12 Try to construct `a < b` where `predicate(a)` is true but `predicate(b)` is false. If one exists, first-true binary search is invalid.
- 13 Equal boundary and middle values can hide which half is sorted. Safely shrinking boundaries can remove only one or two candidates, producing $O(n)$ worst case.
- 14 Arrays provide $O(1)$ midpoint access. Locating a linked-list midpoint costs traversal, so repeated binary search does not retain $O(\log n)$ total time.
- 15 Use a fixed iteration count tied to required precision, or stop when interval width is below epsilon while guarding stagnation from representable floating-point spacing.
- 16 Search a cut from 0 through the shorter length; the other cut follows from the required left-side size. Missing left values use negative infinity and missing right values use positive infinity. A valid partition satisfies both cross inequalities. Odd total returns the maximum left value; even total averages maximum left and minimum right using widened arithmetic. Time is $O(\log(\min(m, n)))$ and space is $O(1)$.
- 17 Search between the matrix minimum and maximum. Count values at most the candidate in $O(\text{rows} + \text{columns})$ by walking from bottom-left: move right after accepting a column prefix and up when the value is too large.

The count cannot decrease as the candidate increases, so first-true search returns the kth occurrence, including duplicates.

Boundary engineering solutions

- 18 Maintain `[0, left)` strictly below target and `[right, n)` at least target. For target below all, right repeatedly moves to zero; inside, false indexes move left past them while equality stays candidate; above all, left reaches `n`. Termination `left==right` is the insertion boundary.
- 19 Require `low<=high` and `predicate(high)==true`. At middle, true retains `[low,middle]`; false retains `[middle+1,high]`. Use signed overflow-safe average `(low & high) + ((low ^ high) >> 1)`. Monotonicity is a caller precondition, not cheaply verifiable by the helper.
- 20 Search last `m` satisfying `m==0 || m<=value/m`, never `m*m`. Keep a saved answer on true and move left boundary upward; move right downward on false. Cap high at floor sqrt of `Long.MAX_VALUE` or value. Reject negatives.
- 21 With distinct values, one half is sorted. Test target against that half's exact inclusive/exclusive boundaries and discard the other. With `[1,1,1,0,1]`, left/middle/right equality gives no information; shrinking duplicates may be necessary and worst case becomes linear.
- 22 Validate every row has the first row's column count and handle zero columns. Search flat half-open range `[0, (long) rows*columns)`, mapping by division/remainder. This assumes each row and cross-row boundary form one globally sorted row-major sequence.
- 23 Generate small values from a narrow range, sort, and linearly advance expected lower while `< target`, then expected upper while `<= target`. Compare both functions across empty, duplicate-heavy, and absent targets with a fixed reproducible seed.

SDE-2 CORE CHAPTER

Chapter 9 - Executable Binary-Search Boundary Checks

This dependency-free Java 21 companion validates signed-safe bounds, integer square root, rotated search, answer search, long-index matrix search, termination, and differential oracles. A successful run prints PASS 18 binary-search checks.

JAVA (continued 1/8)

```
import java.util.Arrays;
import java.util.Random;
import java.util.function.LongPredicate;

public final class BinarySearchInterviewChecks {
    private BinarySearchInterviewChecks() {}

    static int lowerBound(int[] sorted, int target) {
        int left = 0;
        int right = sorted.length;
        while (left < right) {
            int middle = left + (right - left) / 2;
            if (sorted[middle] < target) {
                left = middle + 1;
            } else {
                right = middle;
            }
        }
        return left;
    }

    static int upperBound(int[] sorted, int target) {
        int left = 0;
        int right = sorted.length;
        while (left < right) {
            int middle = left + (right - left) / 2;
            if (sorted[middle] <= target) {
                left = middle + 1;
            } else {
                right = middle;
            }
        }
        return left;
    }
}
```

JAVA (continued 2/8)

```
static int[] equalRange(int[] sorted, int target) {
    int first = lowerBound(sorted, target);
    if (first == sorted.length || sorted[first] != target) {
        return new int[] {-1, -1};
    }
    return new int[] {first, upperBound(sorted, target) - 1};
}

/** Finds the first true value in a closed domain whose high endpoint is true. */
static long firstTrue(long low, long high, LongPredicate predicate) {
    if (low > high || !predicate.test(high)) {
        throw new IllegalArgumentException("nonempty domain with true high required");
    }
    while (low < high) {
        long middle = (low & high) + ((low ^ high) >> 1);
        if (predicate.test(middle)) {
            high = middle;
        } else {
            low = middle + 1;
        }
    }
    return low;
}

static long minimumCapacity(int[] weights, int days) {
    if (weights.length == 0 || days <= 0) {
        throw new IllegalArgumentException("invalid input");
    }
    long left = 0L;
    long right = 0L;
    for (int weight : weights) {
        if (weight <= 0) {
```

JAVA (continued 3/8)

```
        throw new IllegalArgumentException("positive weights required");
    }
    left = Math.max(left, weight);
    right += weight;
}
while (left < right) {
    long middle = left + (right - left) / 2L;
    if (canShip(weights, days, middle)) {
        right = middle;
    } else {
        left = middle + 1L;
    }
}
return left;
}

private static boolean canShip(int[] weights, int days, long capacity) {
    int usedDays = 1;
    long load = 0L;
    for (int weight : weights) {
        if (load + weight > capacity) {
            usedDays++;
            load = 0L;
        }
        load += weight;
    }
    return usedDays <= days;
}

static long integerSquareRoot(long value) {
    if (value < 0) {
        throw new IllegalArgumentException("value cannot be negative");
    }
}
```

JAVA (continued 4/8)

```
    long left = 0;
    long right = Math.min(value, 3_037_000_499L);
    long answer = 0;
    while (left <= right) {
        long middle = left + (right - left) / 2L;
        if (middle == 0 || middle <= value / middle) {
            answer = middle;
            left = middle + 1L;
        } else {
            right = middle - 1L;
        }
    }
    return answer;
}

static int searchRotatedDistinct(int[] values, int target) {
    int left = 0;
    int right = values.length - 1;
    while (left <= right) {
        int middle = left + (right - left) / 2;
        if (values[middle] == target) {
            return middle;
        }
        if (values[left] <= values[middle]) {
            if (values[left] <= target && target < values[middle]) {
                right = middle - 1;
            } else {
                left = middle + 1;
            }
        } else if (values[middle] < target && target <= values[right]) {
            left = middle + 1;
        } else {
            right = middle - 1;
        }
    }
}
```

JAVA (continued 5/8)

```
    }  
  }  
  return -1;  
}  
  
static int minimumEatingSpeed(int[] piles, int hours) {  
  if (piles.length == 0 || hours < piles.length) {  
    throw new IllegalArgumentException("at least one hour per nonempty pile required");  
  }  
  int maximum = 0;  
  for (int pile : piles) {  
    if (pile <= 0) {  
      throw new IllegalArgumentException("positive piles required");  
    }  
    maximum = Math.max(maximum, pile);  
  }  
  long answer = firstTrue(1, maximum, speed -> canEat(piles, hours, speed));  
  return Math.toIntExact(answer);  
}  
  
private static boolean canEat(int[] piles, int hours, long speed) {  
  long used = 0;  
  for (int pile : piles) {  
    used += (pile + speed - 1L) / speed;  
    if (used > hours) {  
      return false;  
    }  
  }  
  return true;  
}  
  
static boolean searchRowMajorMatrix(int[][] matrix, int target) {  
  if (matrix.length == 0) {
```

JAVA (continued 6/8)

```
        return false;
    }
    int columns = matrix[0].length;
    for (int[] row : matrix) {
        if (row.length != columns) {
            throw new IllegalArgumentException("rectangular matrix required");
        }
    }
    if (columns == 0) {
        return false;
    }
    long left = 0;
    long right = (long) matrix.length * columns;
    while (left < right) {
        long middle = left + (right - left) / 2L;
        int row = (int) (middle / columns);
        int column = (int) (middle % columns);
        if (matrix[row][column] < target) {
            left = middle + 1L;
        } else {
            right = middle;
        }
    }
    if (left == (long) matrix.length * columns) {
        return false;
    }
    return matrix[(int) (left / columns)][(int) (left % columns)] == target;
}

private static boolean boundsMatchLinearOracle() {
    Random random = new Random(59L);
    for (int trial = 0; trial < 2_000; trial++) {
        int[] values = new int[random.nextInt(50)];
    }
}
```


JAVA (continued 7/8)

```
        for (int index = 0; index < values.length; index++) {
            values[index] = random.nextInt(31) - 15;
        }
        Arrays.sort(values);
        int target = random.nextInt(41) - 20;
        int expectedLower = 0;
        while (expectedLower < values.length && values[expectedLower] < target) {
            expectedLower++;
        }
        int expectedUpper = expectedLower;
        while (expectedUpper < values.length && values[expectedUpper] <= target) {
            expectedUpper++;
        }
        if (lowerBound(values, target) != expectedLower
            || upperBound(values, target) != expectedUpper) {
            return false;
        }
    }
    return true;
}

private static void expectFailure(Runnable action) {
    try {
        action.run();
    } catch (IllegalArgumentException expected) {
        return;
    }
    throw new AssertionError("expected IllegalArgumentException");
}

private static void check(boolean condition, String message) {
    if (!condition) {
        throw new AssertionError(message);
    }
}
```

JAVA (continued 8/8)

```

    }
}

public static void main(String[] args) {
    int[] sorted = {1, 2, 2, 2, 5};
    check(lowerBound(sorted, 2) == 1, "lower bound");
    check(upperBound(sorted, 2) == 4, "upper bound");
    check(Arrays.equals(equalRange(sorted, 2), new int[] {1, 3}), "equal range");
    check(Arrays.equals(equalRange(sorted, 9), new int[] {-1, -1}), "missing");
    check(minimumCapacity(new int[] {1, 2, 3, 1, 1}, 4) == 3L, "capacity");
    check(firstTrue(0, 100, value -> value >= 37) == 37, "first true");
    expectFailure(() -> firstTrue(0, 10, value -> false));
    check(integerSquareRoot(0) == 0 && integerSquareRoot(15) == 3,
        "integer square root basics");
    check(integerSquareRoot(Long.MAX_VALUE) == 3_037_000_499L,
        "integer square root overflow boundary");
    expectFailure(() -> integerSquareRoot(-1));

    int[] rotated = {4, 5, 6, 7, 0, 1, 2};
    check(searchRotatedDistinct(rotated, 0) == 4, "rotated search");
    check(searchRotatedDistinct(rotated, 3) == -1, "rotated missing");
    check(minimumEatingSpeed(new int[] {3, 6, 7, 11}, 8) == 4,
        "binary search on answer");
    expectFailure(() -> minimumEatingSpeed(new int[] {3, 6}, 1));
    check(searchRowMajorMatrix(new int[][] {{1, 3, 5}, {7, 9, 11}}, 7),
        "flattened matrix search");
    check(!searchRowMajorMatrix(new int[][] {{1, 3, 5}, {7, 9, 11}}, 6),
        "matrix missing");
    expectFailure(() -> searchRowMajorMatrix(new int[][] {{1}, {2, 3}}, 2));
    check(boundsMatchLinearOracle(), "bound differential test");
    System.out.println("PASS 18 binary-search checks");
}
}

```

Part II - Practice and Handoff

TRY IT | Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: exact match and insertion point
- Interview Core: first/last occurrence and rotated arrays
- SDE-2 Follow-up: binary search on answer
- Challenge: design and prove a monotone feasibility predicate

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

[Previous book: DSA 11 - Stacks, Queues, Deques, and Monotonic Patterns](#)

[Series index](#)

[Next book: DSA 13 - Trees, Binary Search Trees, and Tries](#)

TRY IT | Completion check

- ☐ I can recognize the core patterns without being told the data structure or algorithm.
- ☐ I can implement the representative Java templates from memory.
- ☐ I can explain why each implementation is correct.
- ☐ I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- ☐ I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that segment in order. The same series codes and positions appear on the website, PDF covers, and canonical download folders.

SEGMENT	BOOK	FOCUSED LEARNING PATH
Java	JAVA 01	Java Foundations for Problem Solving
Java	JAVA 02	Git and GitHub for Java Engineers
Java	JAVA 03	Maven and Gradle for Java Engineers
Java	JAVA 04	Language, OOP, and Modern Java
Java	JAVA 05	Collections, Streams, and I/O
Java	JAVA 06	JVM and Java Execution
Java	JAVA 07	Java Concurrency and the Memory Model
Java	JAVA 08	Performance, Diagnostics, and GC Incidents
Java	JAVA 09	Java Question Bank, Study Plan, and Reference
DSA	DSA 01	Time and Space Complexity for Java Interviews
DSA	DSA 02	Number Systems and Math Foundations for DSA Interviews
DSA	DSA 03	Number Systems Interview Patterns and Rapid Revision
DSA	DSA 04	Bit Manipulation in Java
DSA	DSA 05	Loop Mastery, Patterns, and Index Calculations
DSA	DSA 06	Arrays and Array Problem-Solving Patterns
DSA	DSA 07	Strings and String Problem-Solving Patterns
DSA	DSA 08	Hashing: Maps, Sets, Frequency, and Prefix State
DSA	DSA 09	Recursion and Backtracking in Java
DSA	DSA 10	Linked Lists: Pointer Reasoning and Mutation
DSA	DSA 11	Stacks, Queues, Deques, and Monotonic Patterns
DSA	DSA 12	Binary Search: Bounds, Answers, and Invariants
DSA	DSA 13	Trees, Binary Search Trees, and Tries
DSA	DSA 14	Heaps, Priority Queues, Selection, and Top-K
DSA	DSA 15	Graphs: Traversal, Ordering, Paths, and Union-Find
DSA	DSA 16	Greedy Algorithms: Recognition, Proofs, and Scheduling
DSA	DSA 17	Dynamic Programming: State, Transitions, and Optimization
Frameworks	FW 01	MySQL for Java Backend Interviews
Frameworks	FW 02	Hibernate and JPA for Java Backend Interviews
Frameworks	FW 03	Spring Framework for Java Engineers
Frameworks	FW 04	Spring Boot for Java Backend Engineers
Frameworks	FW 05	Spring Boot and REST APIs

SEGMENT	BOOK	FOCUSED LEARNING PATH
Frameworks	FW 06	Spring Data for Java Backend Engineers
Frameworks	FW 07	MongoDB for Java Backend Interviews
Frameworks	FW 08	Redis for Java Backend Interviews
Frameworks	FW 09	Persistence, SQL, JPA, and Caching
Frameworks	FW 10	Apache Kafka and Spring Kafka
Frameworks	FW 11	Spring Ecosystem Extensions
Frameworks	FW 12	Spring AI for Java Engineers
System Design	SD 01	Design, Backend, Testing, and Security
System Design	SD 02	Distributed Systems and System Design
System Design	SD 03	Generative AI System Design

All 40 publication editions are available now. Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that shelf in order. Each deep-dive book remains independently printable and available on the web.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer and the author and editor of the Java SDE-2 Interview Preparation Series. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial approach

Vinay sets the learning sequence and publication standard and performs the final review for Java accuracy, evidence, attribution, and release readiness. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: [LinkedIn](#) | [GitHub](#)

Copyright and Publishing Notes

Copyright 2026 Vinay Reddy Kalluri and credited contributors. Open educational edition.

Book prose, exercises, diagrams, and PDFs are licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). Build scripts and source code are licensed under MIT. Individual credit is recorded in AUTHORS.md and Git history.

Repository: <https://github.com/vinayreddykalluri/SDE2-Interview-Handbook>. Attribution does not imply endorsement. Java and related marks are owned by their respective holders.

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Data Structures and Algorithms - Book 12 of 17 - Study Step DSA-12. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.